IIC2523
Sistemas Distribuidos

Hernán F. Valdivieso López
(2026 - 2 / Clase 19)

Autenticación
¿Qué implicancias tiene este proceso?

Temas de la clase

1. Asegurar identidad y sus desafíos
2. Soluciones a los desafíos de Autenticación
3. Canal seguro
4. Delegación: Introducción a OAuth 2.1

Asegurar identidad y sus desafíos

Asegurar identidad y sus desafíos

La clase pasada vimos la noción de "autenticarse" mediante 2 formas:

- Firmas digitales mediante llave asimétrica.
  - Encriptar mensaje con llave privada.
  - Solo con la llave pública se puede desencriptar.
  - Garantiza que el mensaje fue creado por su dueño.

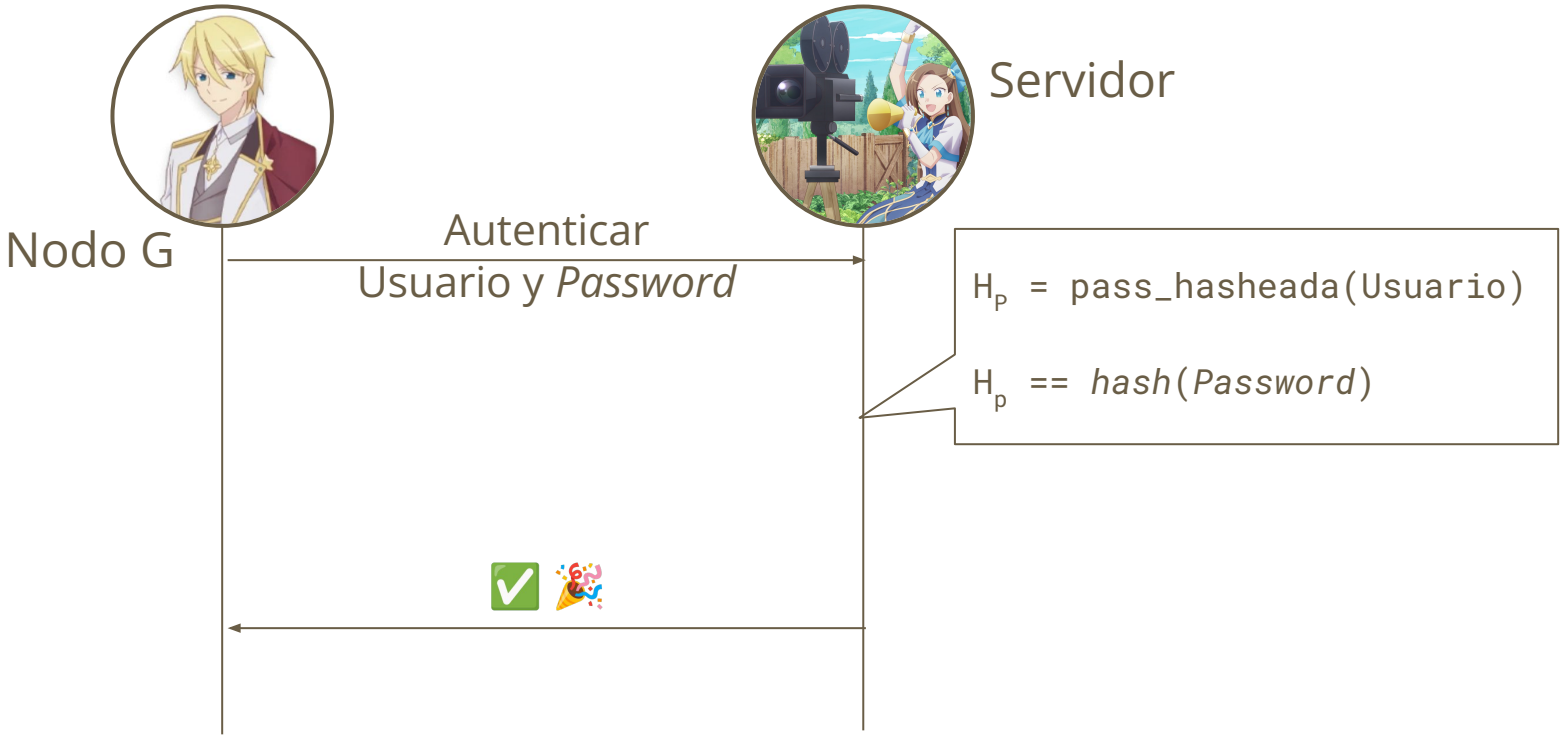
Asegurar identidad y sus desafíos

La clase pasada vimos la noción de "autenticarse" mediante 2 formas:

- Firmas digitales mediante llave asimétrica.
  - Encriptar mensaje con llave privada.
  - Solo con la llave pública se puede desencriptar.
  - Garantiza que el mensaje fue creado por su dueño.
- Firmas digitales usando función *hash*
  - Aplicar *hash* al documento/mensaje.
  - Encriptar el *hash* con llave privada.
  - Solo con la llave pública se puede desencriptar el *hash*.
  - Verificar el *hash* desencriptado con el *hash* del documento para verificar la integridad y autenticidad del mensaje.

Asegurar identidad y sus desafíos

Existe otras formas → Autenticación simple mediante comparación de *hash*.



Asegurar identidad y sus desafíos

Existe otras formas → *Challenge-response protocol*

Asegurar identidad y sus desafíos

Existe otras formas → *Challenge-response protocol*

- Familia de protocolos que permiten la autenticación de entidades.

1. Una parte (verificador) presenta un desafío.
2. La parte que se quiere autenticar recibe el desafío y elabora una respuesta que envía al verificador.
3. El verificador recibe la respuesta y evalúa si la respuesta responde correctamente al desafío.

Asegurar identidad y sus desafíos

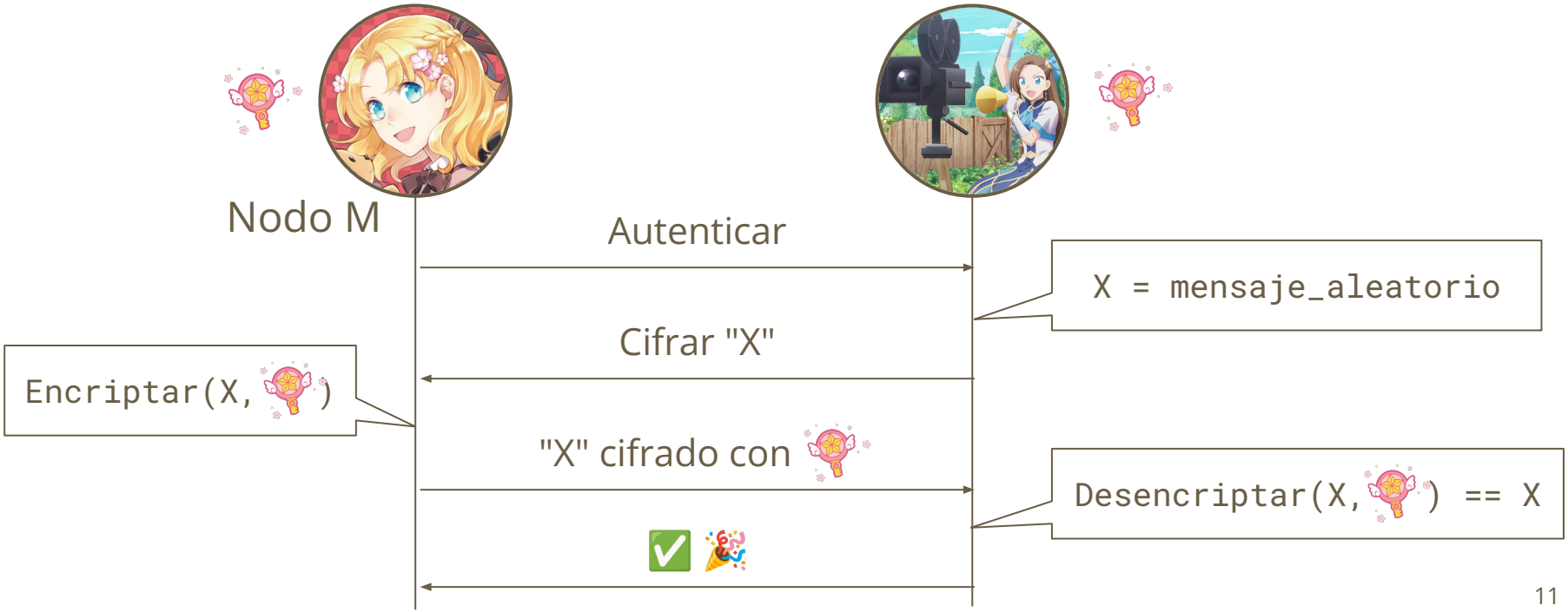
Existe otras formas → *Challenge-response protocol*

- Familia de protocolos que permiten la autenticación de entidades.
  1. Una parte (verificador) presenta un desafío.
  2. La parte que se quiere autenticar recibe el desafío y elabora una respuesta que envía al verificador.
  3. El verificador recibe la respuesta y evalúa si la respuesta responde correctamente al desafío.
- **Se puede realizar aplicando llave simétrica o asimétrica.**
- Un ejemplo es la contraseña de un solo uso para verificación.

Asegurar identidad y sus desafíos

Existe otras formas → *Challenge-response protocol*

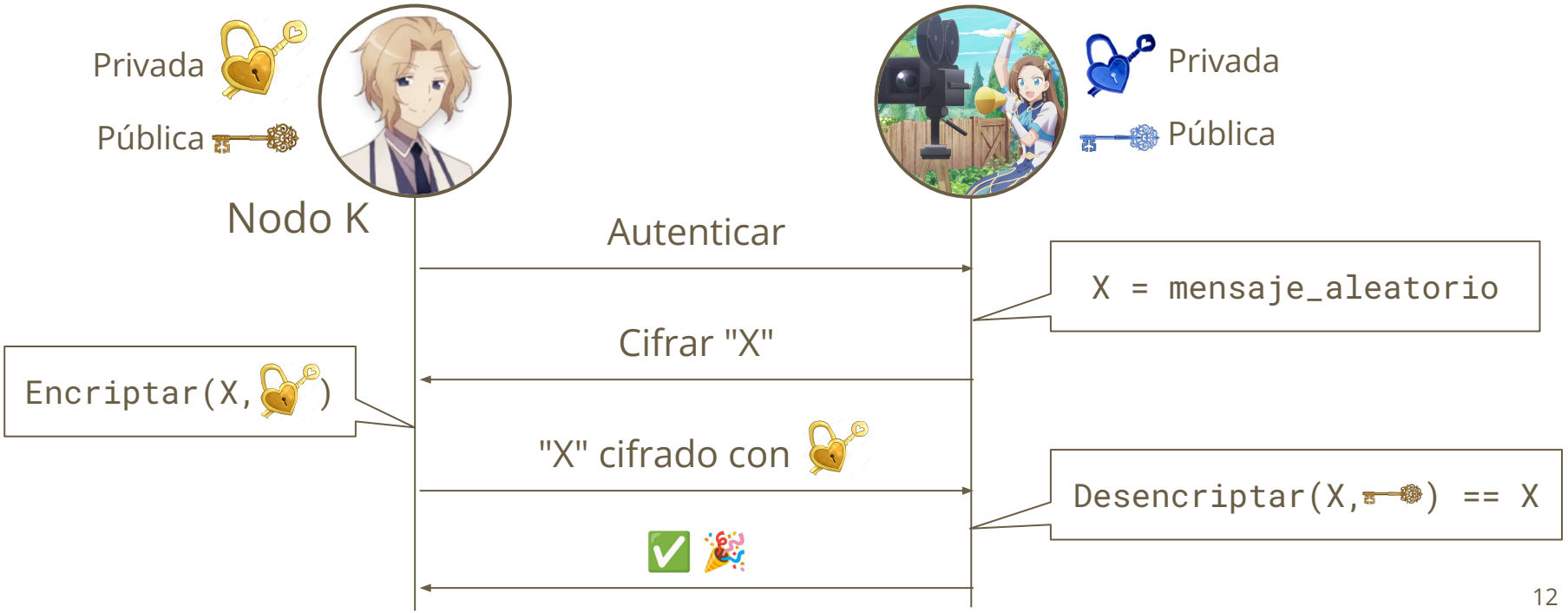
- Se puede realizar aplicando **llave simétrica** o asimétrica.



Asegurar identidad y sus desafíos

Existe otras formas → *Challenge-response protocol*

- Se puede realizar aplicando llave simétrica o **asimétrica**.



Asegurar identidad y sus desafíos

Desafíos

- **Autenticación en ambos sentidos:** no solo el cliente debe autenticarse ante el servidor, sino que el cliente también debe asegurarse de la identidad del servidor.
- **Escalabilidad:** la administración de claves crece a medida que hay más nodos en una red distribuida.
- **Delegación de credenciales:** a veces el usuario desea delegar su identidad o permisos de forma controlada a otro agente sin exponer sus credenciales originales.
- **Expiración**: es necesario tener un control sobre cada cuánto verificar la identidad de una de las partes.

Soluciones a los desafíos de Autenticación

Infraestructura de Clave Pública (PKI)

Kerbero

Infraestructura de Clave Pública (PKI)

- Cada entidad (usuario, servidor o servicio) genera una clave **pública** y **privada**.
- Se crea un certificado que valida la identidad de la entidad.
- Existe una **Autoridad Certificadora (CA)** que firma digitalmente un certificado que vincula una clave pública con la identidad de su dueño.

Infraestructura de Clave Pública (PKI) - Crear

Crear llave privada y pública

Servidor (Juego)

Privada





Pública



CA





Privada



Pública

Infraestructura de Clave Pública (PKI) - Crear

Crear y enviar *Certificate Signing Request* (*CSR*) a la Autoridad Certificadora (CA)



CSR

- Pública
- Información identificatoria
- Información adicional

Infraestructura de Clave Pública (PKI) - Crear

Se realiza un *Certificate Signing Request* (CSR) → CA verifica contenido del certificado

Servidor (Juego)

Privada





Pública



CA

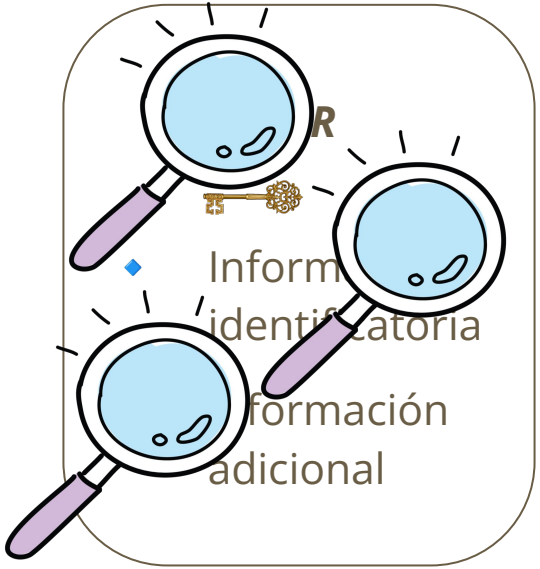




Privada

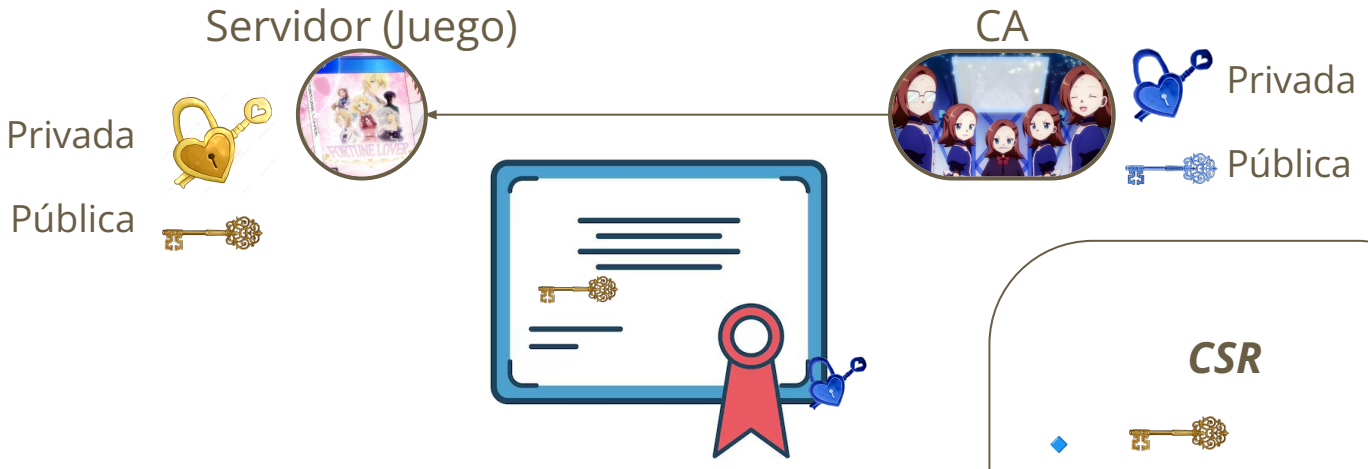


Pública



Infraestructura de Clave Pública (PKI) - Crear

Se firma el contenido del CSR con clave secreta y se emite el certificado.



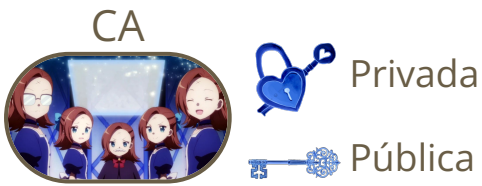
CSR

-
- Información identificatoria
- Información adicional

Infraestructura de Clave Pública (PKI) - Verificar

Cliente recibe certificado

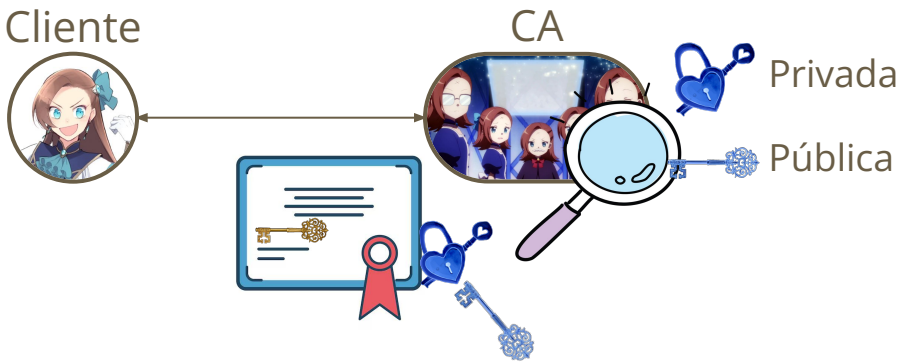




Infraestructura de Clave Pública (PKI) - Verificar

Verifica que certificado sea firmado por CA (usando su llave pública) y que el CA sea confiable





Infraestructura de Clave Pública (PKI) - Verificar

Verifica contenido del certificado (Validez temporal y coincidencia de información)

Servidor (Juego)

Privada



Cliente



CA





Privada

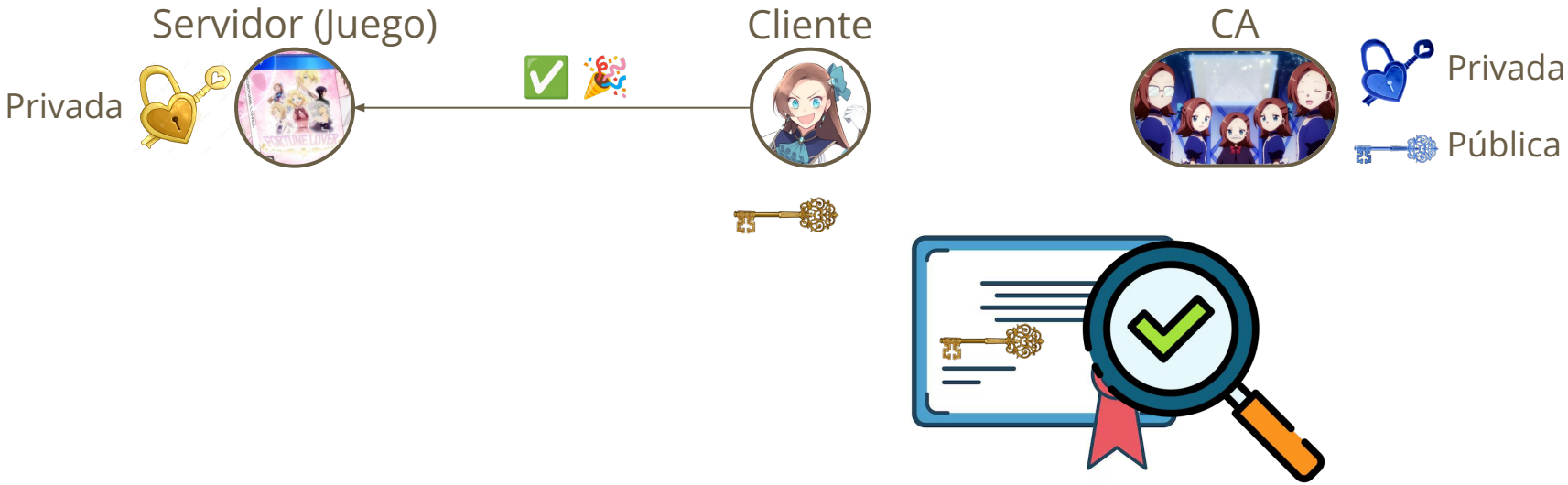


Pública



Infraestructura de Clave Pública (PKI) - Verificar

Extrae llave pública y reconoce la autenticidad del servidor



Infraestructura de Clave Pública (PKI)

Ventaja clave

- Escala eficientemente a muchos nodos: no requiere que cada par de entidades comparta una clave secreta diferente.
- Permite autenticación del servidor (y eventualmente del cliente) sin contacto previo, usando únicamente el certificado firmado por una CA de confianza.

Infraestructura de Clave Pública (PKI) - Noticia

Lo que el compromiso de la clave de CA de eMudhra nos enseña sobre la agilidad de las CA | Consultoría en cifrado

- Llave secreta debía ser un número aleatorio de 384 bits.... no un número aleatorio entre ese rango de valores (entre 0 y 384)... se descubrió que uno de ellos era 4.
- Cualquier usuario podría usar la llave secreta de eMudhra para crear certificados haciéndose pasar por una certificadora autorizada.

Infraestructura de Clave Pública (PKI) - Noticia

Lo que el compromiso de la clave de CA de eMudhra nos enseña sobre la agilidad de las CA | Consultoría en cifrado

- Llave secreta debía ser un número aleatorio de 384 bits.... no un número aleatorio entre ese rango de valores (entre 0 y 384)... se descubrió que uno de ellos era 4.
- Cualquier usuario podría usar la llave secreta de eMudhra para crear certificados haciendose pasar por una certificadora autorizada.

Su CA puede irse antes que sus certificados: la salida de SSL/TLS de eMudhra y la importancia de la agilidad de la CA | Consultoría en cifrado

- La empresa se retiró del negocio de certificados

Kerberos – Autenticación centralizada

- Protocolo de autenticación desarrollado en el M.I.T.
- Recurre a criptografía **simétrica**.
- Requiere un servidor central de confianza (*Key Distribution Center*, KDC).
- Es un proceso ***Single Sign-On (SSO)***: permite a los usuarios autenticarse una sola vez con sus credenciales y acceder a múltiples servicios o aplicaciones sin necesidad de volver a escribir su contraseña

Kerberos – Autenticación centralizada

- Es un proceso ***Single Sign-On (SSO)***: permite a los usuarios autenticarse una sola vez con sus credenciales y acceder a múltiples servicios o aplicaciones sin necesidad de volver a escribir su contraseña.

🎫 Tickets

- Emite *tickets* temporales para autenticarse ante servicios.
- Permiten que el usuario se autentique múltiples veces sin re-enviar su contraseña.
- Los servicios no necesitan almacenar una llave secreta por usuario, sino que usan las llaves que el sistema entrega.

Kerberos – Componentes y flujo

Usuario

Simulemos un caso donde queremos pedir una hamburguesa, pero sin dar nuestra identidad al servicio que vende hamburguesas

Service Server

Key Distribution Center, KDC

Kerberos – Componentes y flujo

Usuario



Simulemos un caso donde queremos pedir una hamburguesa, pero sin dar nuestra identidad al servicio que vende hamburguesas



Service Server

Key Distribution Center, KDC

Kerberos – Componentes y flujo

Usuario





Authentication Server



Ticket Granting Server (TGS)

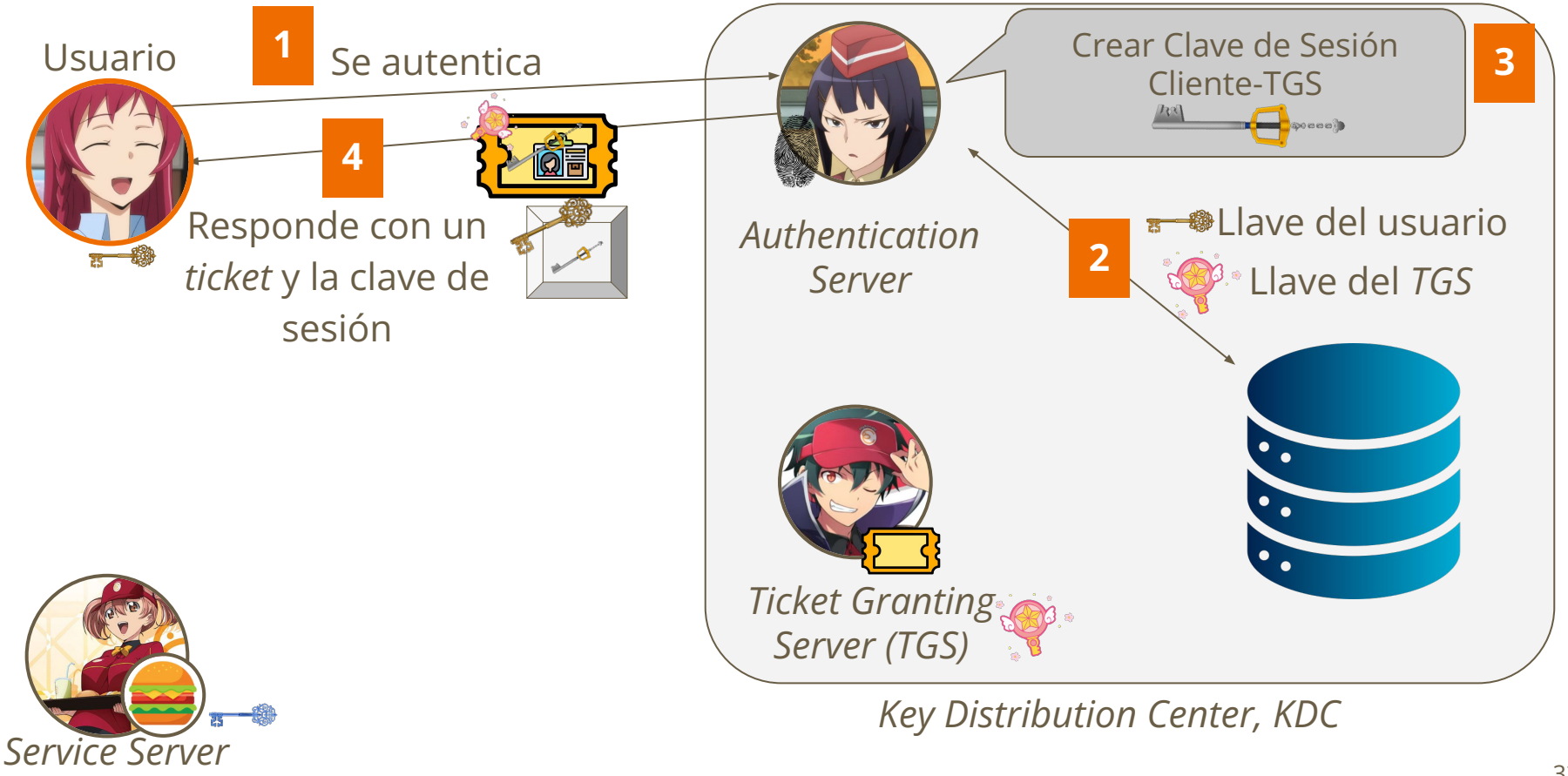


Key Distribution Center, KDC



Service Server

Kerberos – Componentes y flujo

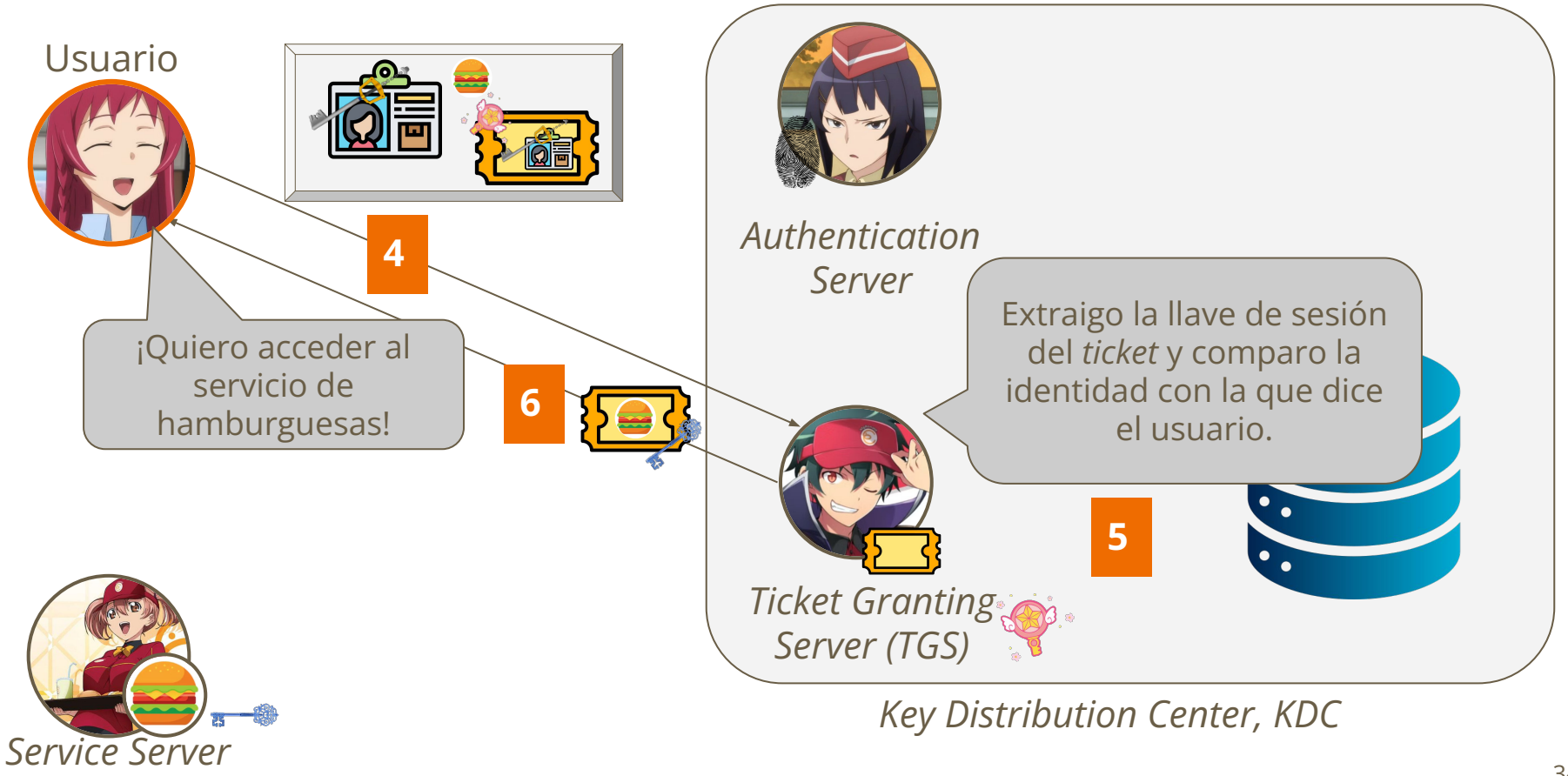


Kerberos – Componentes y flujo

Fase 1: Autenticar al usuario

1. Usuario se autentica ante el AS.
2. *AS* busca la llave secreta del usuario, verifica la identidad del usuario, y si es correcta, busca la llave secreta del *Ticket Granting Server (TGS).*
3. AS genera una Clave de Sesión Cliente-TGS
4. El AS envía al cliente un mensaje que contiene dos elementos clave:
    a. **La Clave de Sesión (Cliente-TGS)**: Cifrada con la clave secreta del usuario. El cliente la descifra con su clave para poder usarla en adelante.
    b. **El *Ticket Granting Ticket* (TGT)**: Cifrado con la clave secreta del TGS. Contiene información de la identidad del usuario y la Clave de sesión. El cliente no puede abrirlo ni modificarlo; solo lo guarda para la siguiente fase.

Kerberos – Componentes y flujo



Kerberos – Componentes y flujo

Fase 2: Solicitar acceso a un servicio

4. Usuario solicita permiso a algún servicio. Para esto, envía un mensaje que contiene dos elementos clave:
    a. **Un autenticador**: Contiene información del usuario y está cifrada con la clave de sesión.
    b. **El *Ticket Granting Ticket* (TGT)** que contiene la clave de sesión, información del usuario y está cifrada con la clave secreta del TGS.
5. El TGS desencripta el TGT para obtener la clave de sesión y la información del usuario. Usa la clave de sesión para abrir el autenticador y valida que la información del usuario que tenía el TGT sea la misma que envió el usuario.
6. Si todo el proceso se logra con éxito, el TGS responde con un *Service Ticket* cifrado con la clave secreta del servidor correspondiente.

Kerberos – Componentes y flujo

Usuario



¡Quiero mi hamburguesa!

8

7







Service Server



Authentication Server



Ticket Granting Server (TGS)



Key Distribution Center, KDC

Kerberos – Componentes y flujo

Fase 3: Solicitar un servicio

7. El cliente usa el *Service Ticket* para solicitar el acceso directamente al servidor correspondiente.
8. El servidor final valida el *Service Ticket* utilizando su propia clave secreta y provee el servicio solicitado.

El *Service Ticket* reemplaza la necesidad que el usuario deba autenticarse con otros servicios y que dichos servidores deban almacenar una llave secreta distinta por usuario.

Kerberos – Componentes y flujo (Resumen)

Fase 1: *Authentication Server (AS)*

1. El usuario solicita acceso al Servidor de Autenticación (AS).
2. El AS verifica su identidad y le devuelve dos elementos: La Clave de Sesión (Cliente-TGS) y el TGT

Fase 2: *Ticket Granting Server (TGS)*

3. El cliente pide permiso para un servicio usando la clave de sesión y el TGT.
4. El TGS valida dicha información.
5. Si coinciden, el TGS responde con un *ticket server* cifrado con la clave del servidor final.

Fase 3: Servidor Final

6. El cliente presenta el *Service Ticket* directamente al servidor final.
7. El servidor lo valida con su propia clave secreta y concede el acceso al servicio solicitado.

Kerberos – Ejemplos reales

- **Active Directory (Microsoft)**
  - Servicio de directorio centralizado para gestionar y proteger los recursos de una red.
  - Un usuario inicia sesión en su computador y accede a carpetas compartidas y otros servicios mediante tickets de Kerberos.

- **Apache Hadoop**
  - *Framework* de código abierto para almacenar y procesar grandes volúmenes de datos de forma distribuida.
  - En un clúster Hadoop configurado en modo seguro, Kerberos permite autenticar tanto a los usuarios como a los servicios del clúster.

Kerberos vs PKI

Criterio	PKI	Kerberos
Criptografía	Asimétrica	Simétrica
Escalabilidad	Muy alta	Moderada (centralizada)
Requiere	Autoridad Certificadora Certificados	Key Distribution Center *Tickets*

Canal seguro

Canales Seguros

- Una capa de servicio sobre un canal de comunicación que garantiza propiedades de seguridad para proteger los datos transmitidos entre dos procesos.
- Usualmente utiliza criptografía simétrica, asimétrica y funciones *hash*.

Canales Seguros

- Una capa de servicio sobre un canal de comunicación que garantiza propiedades de seguridad para proteger los datos transmitidos entre dos procesos.
- Usualmente utiliza criptografía simétrica, asimétrica y funciones *hash*.

Propiedades clave del canal

- **Autenticación**: Identidad fiable de ambas partes.
- **Privacidad**: Cifrado de extremo a extremo.
- **Integridad**: Protección frente a modificaciones.
- ***Anti-replay***: Prevención de retransmisión de mensajes.

Canales Seguros - Fases típicas

1. **Negociación (*handshake*)**

- Intercambio de claves (usualmente con criptografía asimétrica)
- Verificación de identidad (certificados, claves públicas)

2. **Establecimiento del canal**

- Derivación de clave simétrica compartida.
- Selección de algoritmos criptográficos.

3. **Comunicación segura**

- Cifrado y firma de cada mensaje.
- Inclusión de *timestamps* o números de secuencia para evitar *replay*.

Canales Seguros - Ejemplos

Protocolo / Tecnología	Usado en...
HTTPS (TLS)	Web, REST APIs
SSH	Administración remota de servidores
IPsec	VPNs a nivel de red
gRPC + TLS	Comunicación entre microservicios
SMTP + TLS	Correo seguro

Introducción a OAuth 2.1

Introducción a OAuth 2.1

- Delegación de autenticidad consiste en una técnica que permite a una entidad(o un proceso) realizar una acción con la autoridad de otra entidad.

Introducción a OAuth 2.1

- Delegación de autenticidad consiste en una técnica que permite a una entidad(o un proceso) realizar una acción con la autoridad de otra entidad.

OAuth 2.0 (*Open Authorization*):

- Comenzó en noviembre de 2006 con la versión 1.0.
- Protocolo de delegación ampliamente utilizado que permite que las aplicaciones accedan a recursos protegidos en nombre de un usuario, sin necesidad de compartir las credenciales del usuario con la aplicación
  - Iniciar sesión con Google.
  - Iniciar sesión con Github.

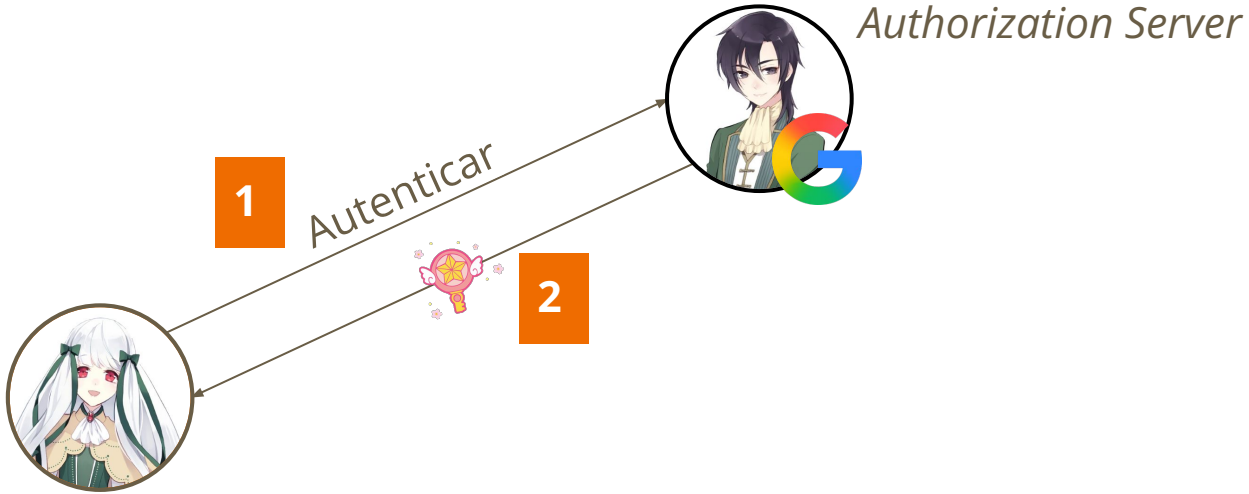
Introducción a OAuth 2.1

- Delegación de autenticidad consiste en una técnica que permite a una entidad(o un proceso) realizar una acción con la autoridad de otra entidad.

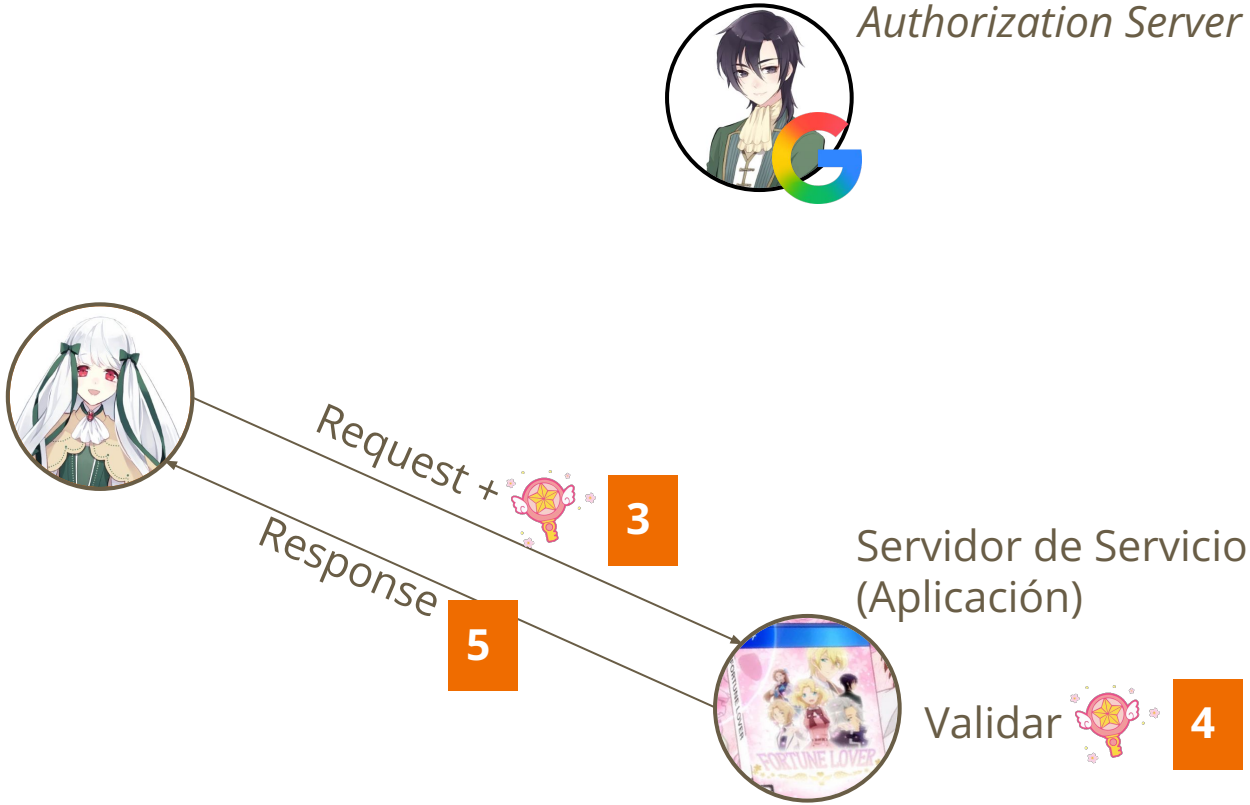
OAuth 2.0 (*Open Authorization*):

- Comenzó en noviembre de 2006 con la versión 1.0.
- Protocolo de delegación ampliamente utilizado que permite que las aplicaciones accedan a recursos protegidos en nombre de un usuario, sin necesidad de compartir las credenciales del usuario con la aplicación
  - Iniciar sesión con Google.
  - Iniciar sesión con Github.
- La versión OAuth 2.1 es la actualización actual y restringe el flujos de autorización, pero la más utilizada hasta ahora es la 2.0.

Introducción a OAuth 2.1 - *Pipeline*



Introducción a OAuth 2.1 - *Pipeline*



Poniendo a prueba lo que hemos aprendido 👀

CineTV, una empresa de *streaming*, implementa una plataforma donde los usuarios deben iniciar sesión. Para esto, los usuarios pueden usar su cuenta de *Spotify* como medio de acceso para evitar ingresar directamente sus credenciales en el sitio.

Finalmente, los usuarios se han comenzado a quejar porque al momento de actualizar el país desde donde se están conectando, ya no pueden ver algunos videos que empezaron en su país de origen.

¿Cuál de las siguientes afirmaciones es **correcta** según lo descrito en este caso?

a. Spotify es un ejemplo de una Autoridad Certificadora para validar la identidad de CineTV.

b. La queja de los usuarios proviene que su identidad no se puede validar en todos los países.

c. CineTV permite OAuth y los permisos a ciertas funcionalidades varían según información del usuario.

d. Spotify es un ejemplo de *Ticket Granting Server* porque da un *token* que permite al usuario ingresar al servicio de CineTV.

e. Ninguna de las anteriores.

Poniendo a prueba lo que hemos aprendido 👀

CineTV, una empresa de *streaming*, implementa una plataforma donde los usuarios deben iniciar sesión. Para esto, los usuarios pueden usar su cuenta de *Spotify* como medio de acceso para evitar ingresar directamente sus credenciales en el sitio.

Finalmente, los usuarios se han comenzado a quejar porque al momento de actualizar el país desde donde se están conectando, ya no pueden ver algunos videos que empezaron en su país de origen.

¿Cuál de las siguientes afirmaciones es **correcta** según lo descrito en este caso?

a. Spotify es un ejemplo de una Autoridad Certificadora para validar la identidad de CineTV.

b. La queja de los usuarios proviene que su identidad no se puede validar en todos los países.

c. CineTV permite OAuth y los permisos a ciertas funcionalidades varían según información del usuario.

d. Spotify es un ejemplo de *Ticket Granting Server* porque da un *token* que permite al usuario ingresar al servicio de CineTV.

e. Ninguna de las anteriores.

Próximos eventos

Próximas clases

- Lunes: no veremos nueva materia. Revisaremos el último enunciado sobre **Hito 3 y 4 del proyecto**.
- Miércoles: Monitoreo de Seguridad. ¿Qué hacer cuando encriptar y garantizar identidad no es suficiente?

Evaluación

- **Control 19 - Opcional** - 2 preguntas sobre la clase de hoy. Se puede responder hasta este viernes a las 20:00.
- **Hito 2 del proyecto** se entrega hoy, máximo el viernes con atraso.
- El viernes se publica el enunciado de Hito 3 y 4 del proyecto.

IIC2523
Sistemas Distribuidos

Hernán F. Valdivieso López
(2026 - 2 / Clase 19)

Créditos (animes utilizados)

Otome Gēmu no Hametsu Furagu Shika Nai Akuyaku Reijō ni Tensei Shiteshimatta...



Hataraku Maō-sama!

